Modeling of the experimental part

1. Goal of the experiments

We want to see if our modified PID controller keeps the water level close to the set point and moves the valve less than a normal PID, when the inflow and outflow are unknown. We also want to know what each extra part of the controller does.

2. The simulated process

The process is a tank with a valve. We use a simple model of the water balance, and it is the model of the class TankSimulator.

The variables are:

- PV(k): the water level at step k, in percent of the tank (from 0 to 100).
- SP(k): the set point, the level we want.
- u(k): the valve position, from 0 to 100.
- u0(k): the null position of the valve, where the inflow is equal to the outflow, so the level does not change. The controller does not know it. In S1 it is 50 all the time, and in other situations it can change.
- d(k): a disturbance added to the level, like rain (positive) or evaporation (negative). It is 0 in all the situations, except S4.
- r(k): the gain of the valve at step k, a random number.

We get the model from the water balance. The volume of water changes with the net flow, $A * \frac{dh}{dt} = Q_{in} - Q_{out}$, where A is the area of the tank and h is the level. The valve controls the inflow, $Q_{in} = c * u$, and the outflow Q_{out} is unknown. The null position is $u_0 = \frac{Q_{out}}{c}$, so the net flow is $c * (u - u_0)$. For a step of time Δt, the level changes with $(c * \frac{\Delta t}{A}) * (u - u_0)$. We call $r = c * \frac{\Delta t}{A}$ and we get the equation of the program. It is an integrator model, like the integrator models used for canal pools in [1] and [15].

$$PV(k+1) = PV(k) + r(k) * (u(k) - u_0(k)) + d(k)$$

The unknown inflow and outflow are inside u_0 and r. When they change, u_0 changes. What we know about the process:

- The level is constant only when $u(k) = u_0(k)$ and $d(k) = 0$. So the controller has to find u_0, and it does not know it.
- $r(k)$ is a whole number from 5 to 39, divided by 100, new at every step. It has 35 possible values with the same chance, so its mean is 0.22 and its standard deviation is about 0.10.
- In one step the level changes with maximum 0.39 · max(u_0, 100 − u_0), which is 19.5 for u_0= 50, plus the disturbance.

- The valve moves maximum 3% in one step: $|u(k) - u(k-1)| \leq 3$.
- If the new level would be outside 0 to 100, the program ignores that update and keeps the old level.
- The process runs on a timer, one step every 150 ms, and we count time in steps.
- The program starts with $PV(0) = 10,\ u(0) = 50$ and SP = 74.

We assume that the flow through the valve is linear around u_0, that $r(k)$ is independent from one step to the next, that the sensor reads the level exactly (we add noise only in situation S5), and that the controller makes one decision for each step of the process.

3. The controller

The controller makes one decision at every step. We write the steps in the order of the code (class ExtendedPid).

1. Error and dead-band: $e(k) = PV(k) - SP(k)$, in percent of the range (the range is 100). If e is positive, the level is too high. If $|e(k)|$ is smaller than the dead-band (0.2), the controller takes $e(k) = 0$, so it does not react to tiny errors.
2. Smoothing of the error: $e_s(k) = \frac{e_s(k-1) + e(k)}{2}$, with $e_s(0) = e(0)$.
3. Derivatives: $d_1(k) = e(k) - e_s(k-1)$and $d_2(k) = d_1(k) - d_{1s}(k-1)$, both rounded to 5 decimals. They are smoothed in the same way: $d_{1s}(k) = \frac{d_{1s}(k-1) + d_1(k)}{2}$ and $d_{2s}(k) = \frac{d_{2s}(k-1) + d_2(k)}{2}$. At the start $d_1(0) = d_2(0) = 0$ and $d_{1s}(-1) = d_{2s}(-1) = 0$. The steps are made by a timer at equal time intervals, so $dt = 1$ and we do not divide by time. The second derivative needs at least 3 values of the error, so in the first 5 steps the controller only collects data and the valve stays at 50.
4. Integral: $S(k) = S(k-1) + e(k)$, the right rectangle rule with $dt = 1$ and $S(-1) = 0$. S is limited to ±1500. If S > 0 and e < 0, or S < 0 and e > 0, we reset S to 0. For more precision we could use the trapezoid rule, $S(k) = S(k-1) + \frac{e(k) + e(k-1)}{2}$.The program has a setting for it, but we use the rectangle rule.
5. Derivative part: $D(k) = \frac{3 * d_{1s}(k) + 2 * d_{2s}(k)}{5}$. It uses $\frac{3}{5}$ of the rate and $\frac{2}{5}$ of the acceleration.
6. The three functions: $f(x) = sign(x) * \frac{ln(1 + k * |x|)}{k}$, with k = 0.1 for f_p and k = 0.01 for f_i. The third function is $f_d(x) = x$, without compression.
7. Output: the controller computes $v(k)$ and asks for the valve position $u^*(k)$, limited between 0 and 100. The valve goes there with a maximum 3% per step.

$$v(k) = -(K_p f_p(e_s(k)) + K_i f_i(S(k)) + K_d f_d(D(k))),\ u^*(k) = 50 + 0.5v(k)$$

The weights are K_p = 10, K_i = 0.2 and K_d = 25, and 50 is the bias of the controller. We found the weights with the tuning from chapter 8.

Two more rules:

- Non-convergence: the error moves away from zero when (e > 0 and $d_{1s} > 0$) or (e < 0 and $d_{1s} < 0$). The controller counts the steps in a row like this. When it reaches 5 steps in a row, it cancels the derivative component: $d_{1s} = 0$, $d_{2s} = 0$ and $e_s = e$. This is done after the decision of that step, so it changes the next steps. Then the count starts again.
- Fallback: the fallback to the last convergent valve position is not in the simulation. It is useful only for a real process, where the null position of the valve can really change.

How the functions behave:

x	f(x) = x	$f_p(x)$	$f_i(x)$
1	1.00	0.95	1.00
5	5.00	4.05	4.88
10	10.00	6.93	9.53
50	50.00	17.92	40.55
100	100.00	23.98	69.31

Near zero, both functions are almost the line f(x) = x (the slope is 1). For big values they grow slowly, like a logarithm, so a big error or a big accumulated sum counts less than in the line. The constant k decides how strong this is. The function f_i uses a smaller k, because it is applied to the sum S, which is much bigger than the error. The sum can reach 1500, where $f_i(1500) = 277.3$. We chose this form of function, and we found the constants k together with the weights, in the tuning from chapter 8.

With only the proportional part, the valve is $u^* = 50 - 5 * f_p(e_s)$, where 5 = 0.5 · 10. The level stops changing when $u^* = u_0$. For $u_0 = 62$ this gives $f_p(e_s) = -2.4$, so e_s is about −2.7 and the level stays 2.7 percent below the set point. A valve opening that is proportional to the error can leave the level blocked there. The integral keeps adding the error, so the valve keeps moving until the error is zero, and in this way the controller finds the null position u_0 that it does not know. The derivative limits the movement if the level changes too fast.

4. Data

We do not use an external dataset. The data come from the simulator: at every step it gives the level, the set point and the valve position. Each run has N = 400 steps. A run is defined by the controller, the test situation and a seed. The seed gives the random sequence of r(k) and of the sensor noise, so the same seed gives the same run. For a fair comparison, all controllers use the same seeds. The program has a box for the seed.

5. Test situations and experiments

We use six test situations. In all of them N = 400, PV(0) = 10, u(0) = 50, and the event happens at step 150 (in S1 the event is the start). In the situations we use the model with the disturbance, and the controller reads the level with a possible noise:

$$PV(k+1) = PV(k) + r(k) * (u(k) - u_0(k)) + d(k),\ PV_m(k) = PV(k) + n(k)$$

Here n(k) is a random number with mean 0 and standard deviation σ. The metrics use the true level PV(k), not PV_m(k).

Situation	What happens	SP(k)	u_0(k)	d(k)	σ
S1	Start of the program	74	50	0	0
S2	Unknown inflow change	74	50 until step 149, then 62	0	0
S3	The set point is dragged with the mouse	74 until step 149, then 20	50	0	0
S4	Heavy rain, then strong evaporation	74	50	+4 for steps 150 to 179, −2 for steps 250 to 299, else 0	0
S5	Noise on the level reading	74	50 until step 149, then 62	0	0.5
S6	Extreme unknown flow change	74	50 until step 149, then 75	0	0

We use these values for all the results.

Experiment	What we do	Situations
E1	Compare our controller with P, PI, PID and PID+ (a PID with filtered derivative and anti-windup)	S1 to S6
E2	Noise: random gain of the valve (in all runs) and noise on the level reading	S5
E3	Sudden change of the inflow	S2, S6
E4	Extreme disturbances	S4, S6

Experiment	What we do	Situations
E5	Process that does not converge: we count how many times the derivative is cancelled, with and without the cancel	S4, S6
E6	We turn off one part at a time (second derivative, smoothing, dead-band, integral reset, cancel of the derivative) and we replace f_p, f_i, or both, with f(x) = x	S1 to S6
E7	We change the gains K_p, K_i and K_d, one at a time, by 0.5, 0.8, 1.25 and 2 times	S2, S3, S4
E8	Manual test: we drag the SP with the mouse and we stop the action engine (the box Process runs)	free mode

We added these things to the program:

- a seed for the random generator (a box in the window), so the same sequence can be used for all controllers;
- u_0 separate from the bias, as a slider, so we can change it during the run;
- the disturbance d(k) (rain and evaporation) added to the level;
- noise on the level reading;
- a list with the situations S1 to S6, and a run of 400 steps;
- switches to turn each part of the controller on and off;
- the baseline controllers P, PI, PID and PID+;
- saving of the level, the set point and the valve position at every step in a CSV file;
- the code that computes the metrics from chapter 6, shown at the end of a run;
- a second window that runs E1, E6 and E7 and saves the tables;
- the automatic checks of the simulator (chapter 7) and the tuning of the weights (chapter 8).

6. Metrics

We compute these values for each run. N = 400, and k_0 is the step of the event (0 for S1 and 150 for the others). They use the true error e(k) = PV(k) − SP(k) and the real valve position u(k).

$$IAE = \sum_{k=0}^{N-1} |e(k)| \quad E_{max} = max_{k \geq k_0} |e(k)| \quad VM = \sum_{k=6}^{N-1} |u(k) - u(k-1)|$$

- IAE: the sum of |e(k)| over all steps. It shows the total error.
- Emax: the biggest |e(k)| after the event.
- Settling time $T_s = k_s - k_0$, where k_s is the first step from which |e(j)| ≤ 1 for all j ≥ k_s. If there is no such step, the run is not settled. We give the mean T_s of the settled runs and the percentage of settled runs.
- Overshoot, only when the error at the event is at least 5 (this happens in S1 and S3): with s_0 = sign(e(k_0)), OS = max(0, $max_{k \geq k_0}$ ($-s_0$ * e(k))). It is how much the level goes to the other side of the set point.

- VM: the valve movement, the sum of |u(k) − u(k−1)| from step 6 to the end.
- DC: the number of times the valve changes direction.
- FE: the final error, the mean of |e(k)| over the last 50 steps.
- C: the number of times the derivative component is cancelled in a run.

7. Validation of the simulator

Before the experiments we check that the simulator does what the model says. The program has automatic checks for these things (the button Simulator checks):

- With the valve fixed at u_0 and d = 0, the level does not change.
- With the valve fixed at u_0 + 10, the level goes up by about 2.2 per step on average (0.22 * 10), and with the valve at u_0 − 10 it goes down by about 2.2 per step. We check this over 500 runs of 20 steps.
- The level stays between 0 and 100, and the valve moves at most 3% per step.
- The same seed gives exactly the same run.
- The noise of the sensor has the mean 0 and the standard deviation 0.5 (we check it on 20000 readings).
- For the same seed, the simulator in the experiments and the main window give the same values. The checks show the IAE and the valve movement of seed 1 for each situation. We run the same situation in the main window, with seed 1, and we compare the numbers.

8. Validation of the results and comparison

- Repetition: we run each situation 100 times (seeds 1 to 100) for each controller, with the same seeds for all controllers. We give the mean and the standard deviation of each metric. To compare our controller with another one we use the same seeds, and we count in how many runs ours is better and we give the mean difference.
- Baselines: P, PI and PID are linear and have no extra parts. PID+ is a PID with a filtered derivative (the same smoothing with the previous value, as in our controller) and with anti-windup (we do not add to S when the output is at 0 or 100 and the error would push it further) [7], [11]. They wait the same 5 steps at the start, they use the same output formula as our controller, and their integral is limited to ±1500 like ours.
- Tuning: we tune P, PI, PID and PID+ with a grid search (K_p from 0.5 to 14, K_i from 0.02 to 3.2 and K_d from 0 to 350). We tune our controller with a search that changes one value at a time (K_p, K_i, K_d, the constants k of the three functions and the dead-band) and keeps it if the result is better, in 2 rounds. All of them use the same seeds (1001 to 1020, which we do not use for the results), the same situations (S1 to S6) and the same cost: the mean of IAE + 0.3 * valve movement. The code is in the program (the button Tune the weights).
- The values we found: P with K_p = 6; PI with K_p = 3 and K_i = 0.2; PID with K_p = 14, K_i = 0.02 and K_d = 100; PID+ with K_p = 4, K_i = 0.2 and K_d = 10; our controller with K_p = 10, K_i = 0.2, K_d = 25, k = 0.1 for f_p, k = 0.01 for f_i, k = 0 for f_d and the dead-band 0.2. For the PID, K_p = 14 is at the end of the grid, so a bigger value could be better.

- Ablation: with E6 we see what each part of our controller does.
- When we say that the approach brings a benefit: our controller must have a smaller IAE and Emax than P, PI, PID and PID+ in S2, S4 and S6, a similar or smaller valve movement, and not a worse result in S1. If this is not true, we write the results as they are.
- Literature: the articles we studied use similar metrics: [4] uses MAE and IAE, and [5] uses error indicators and the transition time. They work on canals with a model, so we cannot compare the numbers directly. We compare the methods and the metrics. We expect similar or better error and valve movement when the flows are unknown, because we do not need a model, and we expect a worse precision than the controllers that use a model.

9. Implementation

The program is a C# Windows Forms program (.NET Framework 4.8) made in Visual Studio. The main files are: TankSimulator (the process), ExtendedPid and BasicPid (the controllers), Scenario and Scenarios (the situations), RunLog and RunMetrics (the data and the metrics), Experiments (E1, E6 and E7), SimulatorChecks and Tuner. The main window shows the level, the set point and the valve while it runs, and we use it for the manual test E8. The experiments run on another thread in a second window, and they use the same classes as the main window.

10. Limits of the model

- It is a simulation. We do not use data from a real saltern.
- The model is linear around u_0, and r(k) is random and independent from one step to the next. A real valve and a real tank can be different.
- The model has no time delay, and we assume one decision of the controller for each step of the process.
- The weights of all the controllers are tuned for this simulator and for this cost (IAE + 0.3 * valve movement). On another process, or with another cost, they have to be tuned again. The search changes one value at a time and the grids are not fine, so it can stop at a local minimum.
- The results depend on the six situations we chose.

References used in this chapter

[1] E. Weyer, “Decentralised PI control of an open water channel,” in Proc. 15th IFAC World Congress, Barcelona, Spain, 2002.

[4] K. Shahverdi, A. Noorali, H. Ghodousi, and R. Berndtsson, “Developing internal and external proportional integral derivative water surface controller in HEC-RAS,” Water, vol. 16, no. 12, Art. no. 1699, 2024, doi: 10.3390/w16121699.

[5] X. Li, G. Guan, X. Tian, Y. Luo, Y. Huang, and J. Yang, “Hybrid feedforward-feedback LQR controller based on model prediction for open channel water level control,” J. Hydroinformatics, 2024, doi: 10.2166/hydro.2024.322.

[7] K. J. Åström and T. Hägglund, Advanced PID Control. Research Triangle Park, NC, USA: ISA, 2006.

[11] C. Bohn and D. P. Atherton, “An analysis package comparing PID anti-windup strategies,” IEEE Control Syst. Mag., vol. 15, no. 2, pp. 34–40, 1995.

[15] X. Litrico and V. Fromion, Modeling and Control of Hydrosystems. London, U.K.: Springer, 2009.